

# CLOUD COMPUTING: STORAGE & DATA

## Module 5: Storage and Data Management

Author: Eng. Carlos Andrés Sierra, M.Sc.  
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor  
Computer Engineering Program  
School of Engineering  
Universidad Distrital Francisco José de Caldas

2026-III



UNIVERSIDAD DISTRITAL  
FRANCISCO JOSÉ DE CALDAS

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



# Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases

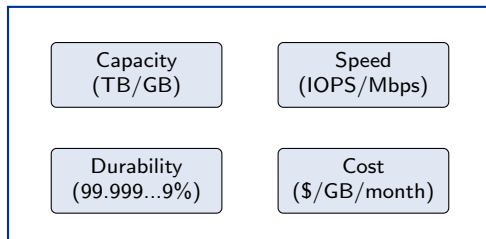


# What Does “Storage” Mean?

## Key Properties

Every **storage system** trades off four properties: **capacity**, **speed**, **durability**, and **cost**.

**Pick 2-3: optimize for your workload**



# Storage Types: Block, File, Object

## Block Storage

Raw disk volumes  
*Fast, structured*

VM disks, Databases

## File Storage

Shared filesystem  
*Concurrent access*

NFS, SMB shares

## Object Storage

Buckets & keys  
*Scalable*

Backups, Archives



# Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- Fast: low latency, high IOPS
- Attached: must be bound to a compute instance
- Limited: prevents instance shutdown (persistent)
- HDD/HUD appears trade cost vs speed
- Supports point-in-time backup

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast:** low latency, high IOPS
- **Attached:** must be bound to a compute instance
- **Durable:** survives instance shutdown (persistent)
- **SSD/HDD options:** trade cost vs. speed
- **Snapshots:** point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast:** low latency, high IOPS
- **Attached:** must be bound to a compute instance
- **Durable:** survives instance shutdown (persistent)
- **SSD/HDD options:** trade cost vs. speed
- **Snapshots:** point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast:** low latency, high IOPS
- **Attached:** must be bound to a compute instance
- **Durable:** survives instance shutdown (persistent)
- **SSD/HDD options:** trade cost vs. speed
- **Snapshots:** point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast:** low latency, high IOPS
- **Attached:** must be bound to a compute instance
- **Durable:** survives instance shutdown (persistent)
- **SSD/HDD options:** trade cost vs. speed
- **Snapshots:** point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# Block Storage

## Definition

Raw **disk volumes** attached to compute instances. Format with any filesystem (ext4, NTFS, etc.). *Think: external hard drive for your VM.*

## Characteristics:

- **Fast**: low latency, high IOPS
- **Attached**: must be bound to a compute instance
- **Durable**: survives instance shutdown (persistent)
- **SSD/HDD options**: trade cost vs. speed
- **Snapshots**: point-in-time backups

## Use Cases

Database storage, transactional workloads, application data requiring low latency.

# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- Shared: multiple VMs read/write simultaneously
- Managed: no filesystem management needed
- Scalable: grows automatically with demand
- Locking: prevents concurrent corruption
- Networking: accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared:** multiple VMs read/write simultaneously
- **Managed:** no filesystem management needed
- **Scalable:** grows automatically with demand
- **Locking:** prevents concurrent corruption
- **Networking:** accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared:** multiple VMs read/write simultaneously
- **Managed:** no filesystem management needed
- **Scalable:** grows automatically with demand
- **Locking:** prevents concurrent corruption
- **Networking:** accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# File Storage

## Definition

**Shared filesystem** (NFS, SMB) accessible from multiple instances simultaneously. *Think: network drive for your team.*

## Characteristics:

- **Shared**: multiple VMs read/write simultaneously
- **Managed**: no filesystem management needed
- **Scalable**: grows automatically with demand
- **Locking**: prevents concurrent corruption
- **Networking**: accessed over network (slightly slower)

## Use Cases

Shared team storage, media files, machine learning datasets, shared caches.



# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- Unlimited capacity: petabytes to exabytes
- Durable: 11 9's (99.999999999%) durability
- Anywhere, anytime, flat namespace, any data type
- Scalability: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Low-cost storage, long-term life

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity:** petabytes to exabytes
- Durable: 11 9's (99.999999999%) durability
- No structure: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- No structure: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- Versioning: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- Lifecycle policies: auto-archive or delete old data
- Cheap: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Object Storage

## Definition

**Unlimited scalability** storage accessed via APIs (*HTTP REST*). Data organized as objects (*key-value pairs*) in **buckets**.

## Characteristics:

- **Unlimited capacity**: petabytes to exabytes
- **Durable**: 11 9's (99.999999999%) durability
- **No structure**: flat namespace, any data type
- **Versioning**: keep old versions of objects
- **Lifecycle policies**: auto-archive or delete old data
- **Cheap**: lowest cost per GB

## Use Cases

Backups, archives, data lakes, static website hosting, media distribution.

# Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables:** rows and columns with schema
- **Primary key:** unique identifier per row
- **Foreign key:** link to other tables
- **ACID:** Atomicity, Consistency, Isolation, Durability
- **Indexes:** speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables**: rows and columns with schema
- Primary key: unique identifier per row
- Foreign key: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- Foreign key: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- ACID: Atomicity, Consistency, Isolation, Durability
- Indexes: speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

### Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

### When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# Relational Databases

## The Model

**Structured data** in **tables** with **relationships** (*foreign keys*). ACID transactions guarantee data consistency.

## Key Concepts:

- **Tables**: rows and columns with schema
- **Primary key**: unique identifier per row
- **Foreign key**: link to another table
- **ACID**: Atomicity, Consistency, Isolation, Durability
- **Indexes**: speed up queries on specific columns

## When to Use

Structured data, complex queries, transactions (banking, ecommerce).



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- Key-Value: Redis — ultra-fast caching
- Document: MongoDB — flexible JSON documents
- Graph: Neo4j — relational graphs and patterns
- Wide-Column: Cassandra — massive distributed data
- Search: Elasticsearch — full-text search

### Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value:** Redis — ultra-fast caching
- **Document:** MongoDB — flexible JSON documents
- **Graph:** Neo4j — relationships and patterns
- **Wide-Column:** Cassandra — massive distributed data
- **Search:** Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value:** Redis — ultra-fast caching
- **Document:** MongoDB — flexible JSON documents
- **Graph:** Neo4j — relationships and patterns
- **Wide-Column:** Cassandra — massive distributed data
- **Search:** Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: ElasticSearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# NoSQL Databases

## Beyond Relational

**Flexible schemas**, horizontal scaling, optimized for specific access patterns (key-value, document, graph, wide-column).

### Types:

- **Key-Value**: Redis — ultra-fast caching
- **Document**: MongoDB — flexible JSON documents
- **Graph**: Neo4j — relationships and patterns
- **Wide-Column**: Cassandra — massive distributed data
- **Search**: Elasticsearch — full-text search

### Trade-off: *CAP Theorem*

Choose 2 of 3: Consistency, Availability, Partition tolerance.



# Outline

- 1 Storage Fundamentals
- 2 Block, File & Object Storage
- 3 Relational & NoSQL Databases



## Recap: Module 5

- **Block Storage:** attached disk volumes, fast, structured access
- **File Storage:** shared filesystem, concurrent access, NFS/SMB
- **Object Storage:** unlimited scalability, durable, cheap (S3, MinIO)
- **Relational:** structured, ACID, complex queries, strong consistency
- **NoSQL:** flexible, scalable, optimized for specific patterns
- **Decision:** choose storage type based on access pattern, scale, consistency needs



# Thanks!

## Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/cloud-computing>

